LAB: Stepper Motor

LAB: Stepper Motor

Date: 2026-10-09

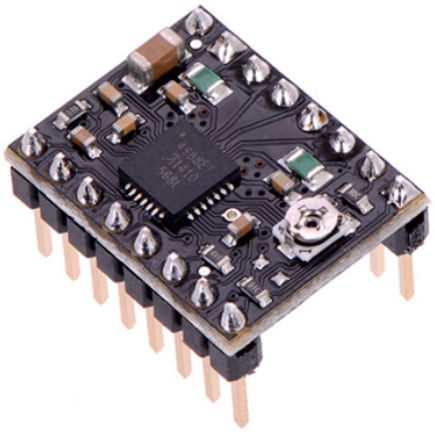
Author/Partner: Juyoung Lee

Demo Video: [EC 26-2] LAB(Stepper Motor)

Tutorial: Stepper Motor Driver (A4988)

A simple method to drive a stepper motor is using a motor driver that receives only a pulse and direction pin.

- **A4988 motor driver** A4988 spec sheet
- **DRV8825**
- Any motor shield based on A4988, DRV8825



A4988 / DRV8825



Driver Shield
based on A4988 / DR8825

The stepper motor for the tutorial can be any unipolar or bipolar (<24V)

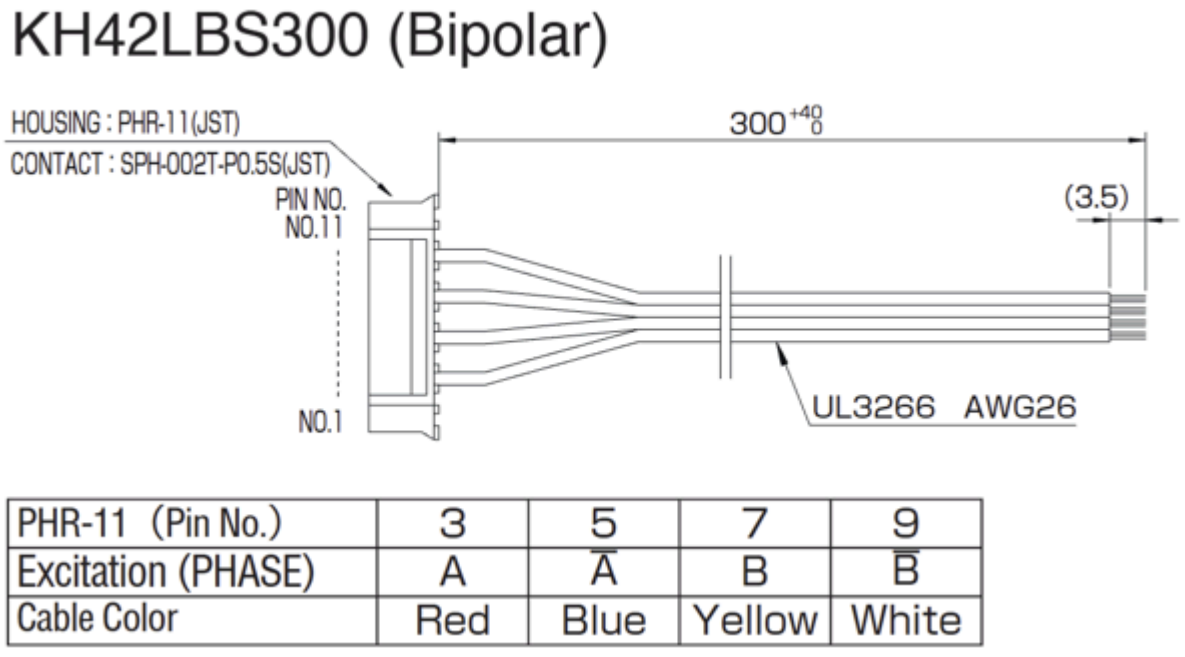
- Stepping Motor (Bipolar 12V, SM-42BYG011-25)
- Stepping Motor (5V, KH4248 b90112:)

Connection Diagram

(1) Connection Diagram (A4988)

1. Connect the motor driver and the stepper motor as follows.
2. Supply 8V from a power supply to VMOT(#16 on Driver)

Functions	Motor	Driver	MCU-Arduino	MCU-STMf411
Step		#7	D2	PA_10
Direction		#8	D3	PB_3
VDD (MCU 5V)		#9	5V	+5V
VDD_GND (MCU GND)		#10	GND	GND
Vmotor (PowerSupply)		#16 to Power(8V), Capacitor 100uF (pin#15-16)		
Vmotor_GND (PowerSupplyGnd)		#15 to Power(GND)		
Motor A, A'	A: RED, A':GRN	#12 to A, #11 to A'		
Motor B, B'	B: YEL, B':BLU	#13 to B, #14 to B'		
Others		#5(RST) to #6(SLP)		
![[Pasted image 20261009220527.png]]				



PHR-11 (Pin No.)	3	5	7	9
Excitation (PHASE)	A	$\overline{A}$	B	$\overline{B}$
Cable Color	Red	Blue	Yellow	White

(2) Connection Diagram (Motor Driver Shield based on A4988)

Pin Connection

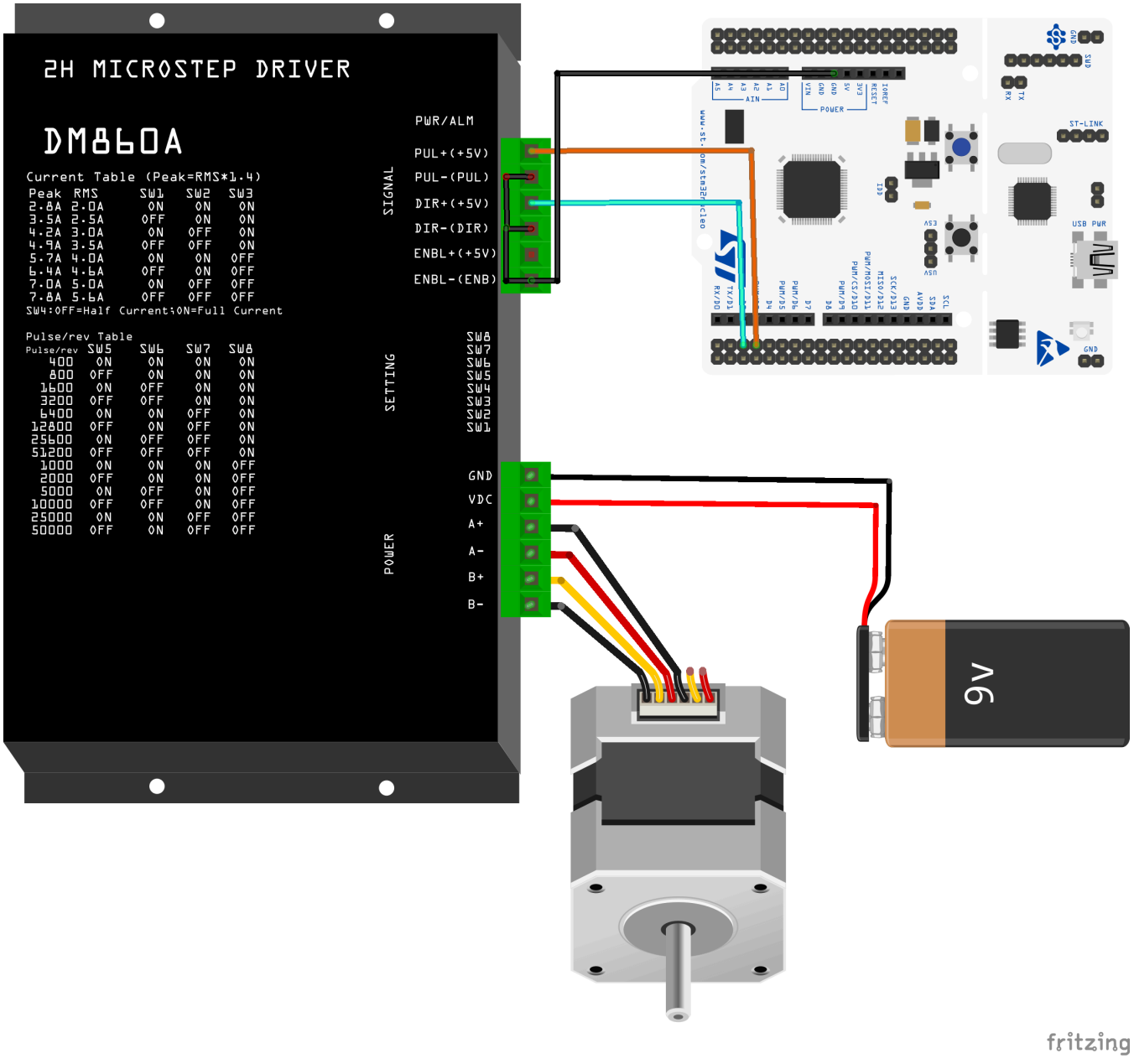
MCU:

- DIR: D2(PA10), STEP: D3(PB3), ENABLE: D6(PB10)

Power Supply: 8~12V

Depends on the motor shield

- SW4 to SW6 : ON (Lowest Current RMS)
- SW1 to SW3: x1 micro-stepping



Tutorial Program

```
#include "stm32f411xe.h"
#include "ecRCC2.h"
#include "ecGPIO2.h"
#include "ecSysTick2.h"

#define MOTOR_DIR PB_3
#define MOTOR_PULSE PA_10

void setup(void)
{
    RCC_PLL_init();
    SysTick_init();
    GPIO_init(MOTOR_DIR, OUTPUT);
    GPIO_init(MOTOR_PULSE, OUTPUT);
}

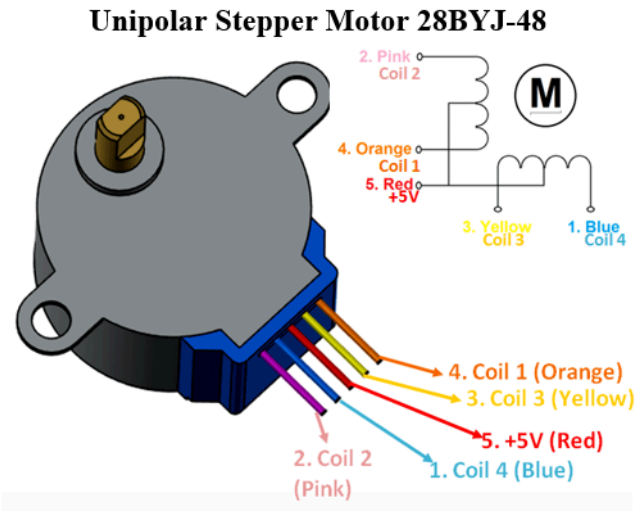

int main(void) {
    setup();
    // Set Dir high (CW)
    GPIO_write(MOTOR_DIR, HIGH);
    while(1){
          for(x = 0; x < 1000; x++)
          {
            GPIO_write(MOTOR_PULSE, HIGH);
            delay_ms(10);
            GPIO_write(MOTOR_PULSE, LOW);
            delay_ms(10);
          }
        delay_ms(1000);
    }
}
```

Problem : Stepper Motor Control with FSM

Problem Description

1. Rotate the stepper motor 5 revolutions with 1 rpm.
2. Repeat the above process in the opposite direction.
3. Increase and decrease the speed of the motor as fast as it rotate to find the maximum and minimum speed of the motor.
4. Apply the half-stepping and repeat the above.

- **Driver: ULN2003 motor driver.**ULN2003 spec sheet
- **Motor:** 28BYJ-48, Search for the spec sheet
- **Number of Step:** 2048 per revolution (Stride step(64) * Gear ratio(32)

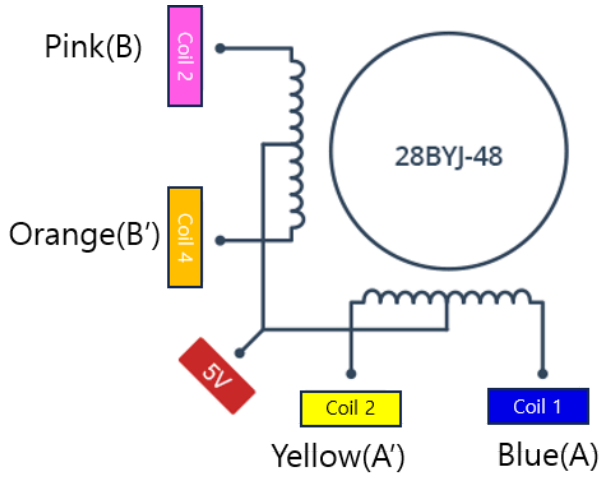


- Rated Voltage: 5V DC
- Number of Phases: 4
- Stride Angle: 5.625°/64
- Gear ratio: 1/32
- Pull in torque: 300 gf.cm
- Coil: Unipolar 5 lead coil

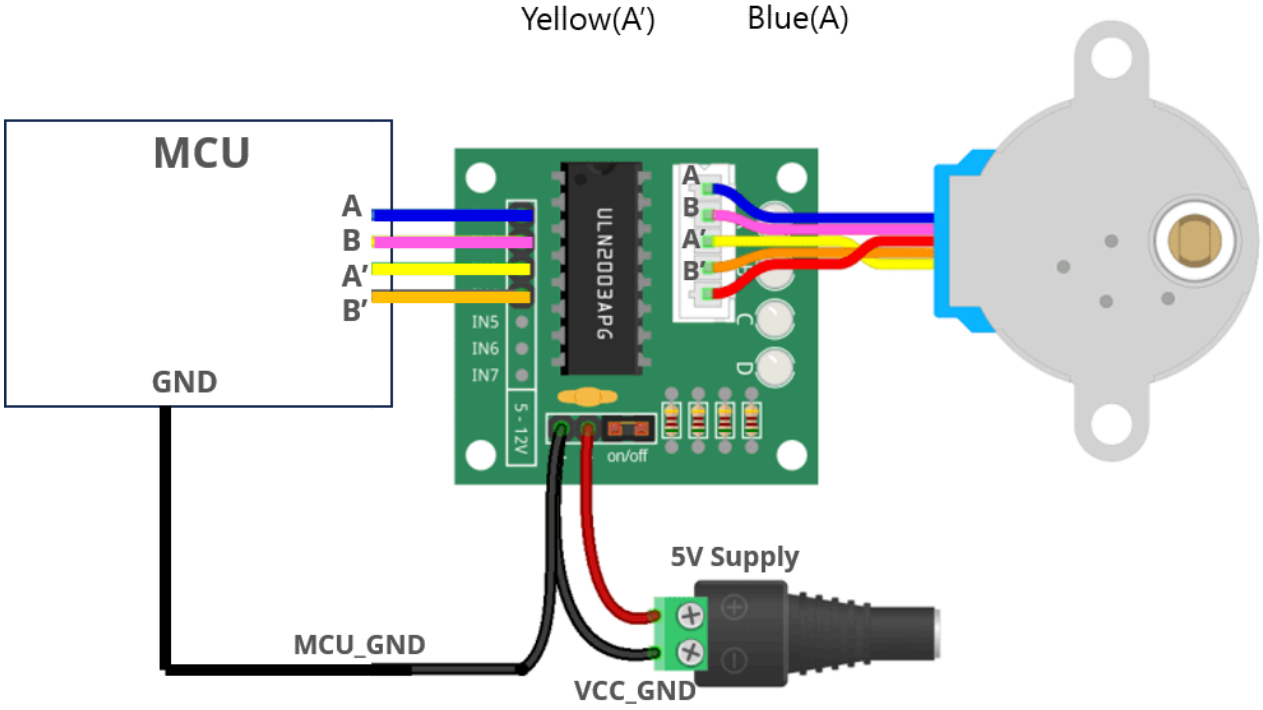
Configuration

![[Pasted image 20261009231659.png]]

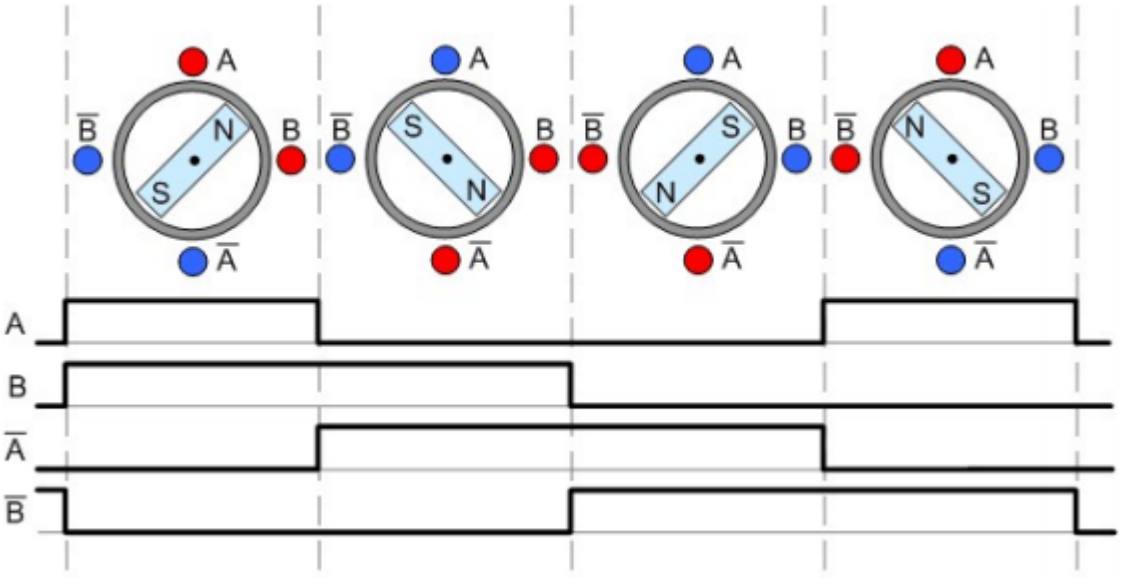
Connection Diagram



28BY –J48
Pin 1: Blue (A)
Pin 2: Pink (B)
Pin 3: Yellow (A')
Pin 4: Orange (B')
Pin 5: Red (common)



FSM design for Stepper Motor Sequence



State Table for Full-Stepping

Full-stepping sequence

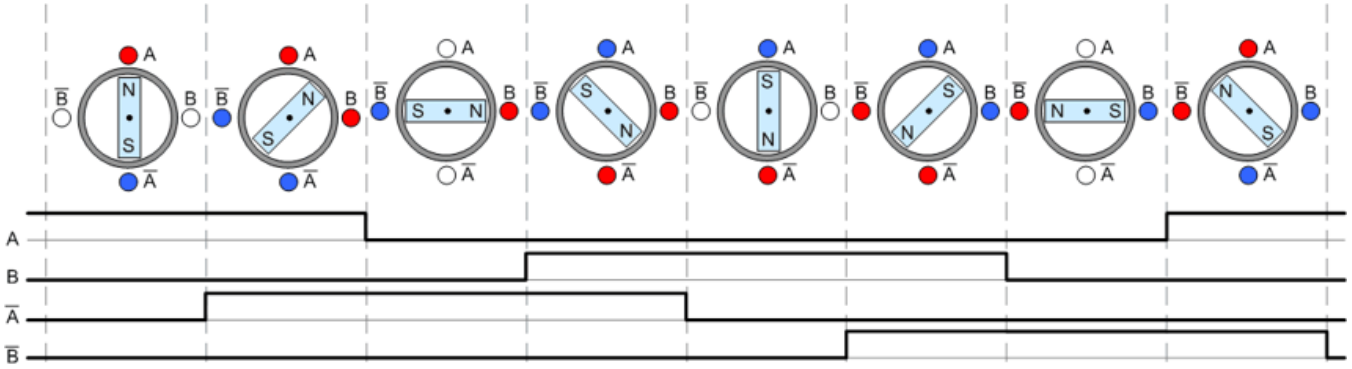
Phase	Port_Pin	Sequence			
		1	2	3	4
A	PB_10	H	L	L	H
B	PB_4	H	H	L	L
A'	PB_5	L	H	H	L
B'	PB_3	L	L	H	H

State Table: Moore FSM

State	Next State		Output
	DIR=0	DIR=1	(A B A′ B′)
S0	S3	S1	1 1 0 0
S1	S0	S2	0 1 1 0
S2	S1	S3	0 0 1 1
S3	S2	S0	1 0 0 1

State Table for Half-Stepping

Half-stepping sequence



PinName	Sequence							
	1	2	3	4	5	6	7	8
A	H	H	L	L	L	L	L	H
B	L	L	L	H	H	H	L	L
A'	L	H	H	H	L	L	L	L
B'	L	L	L	L	L	H	H	H

- **State Table**: Moore FSM

State	Next State		Output
	DIR = 0	DIR = 1	(A B A' B')
S0	S7	S1	1 0 0 0
S1	S0	S2	1 0 1 0
S2	S1	S3	0 0 1 0
S3	S2	S4	0 1 1 0
S4	S3	S5	0 1 0 0
S5	S4	S6	0 1 0 1
S6	S5	S7	0 0 0 1
S7	S6	S0	1 0 0 1

Programming FSM

Preparation: ecStepper2.h
The `ecStepper2.h` header file declares macros, data structures, and function prototypes required for the stepper motor library.

- **Header Protection & Library Includes**
  - `#ifndef __EC_STEPPER2_H` ~ `#endif`: Include guard to prevent multiple inclusion errors during compilation.
    - `ecSTM32F4v2.h`: Includes register definitions, GPIO, EXTI, and SysTick utility functions for the STM32F411 MCU.
- **Macro Definitions (`#define`)**
  - `HALF (0)` / `FULL (1)`: Stepper motor driving mode selection macros (Half-stepping vs. Full-stepping).
  - `DIR_CW (0)` / `DIR_CCW (1)`: Rotation direction macros (Clockwise vs. Counter-Clockwise).
  - `STEP_PER_REV (64*32)`: Total steps per single revolution for the 28BYJ-48 motor (64 stride steps × 32 gear ratio = 2048 steps/rev).
- **Data Types & Structs**
  - `StateNum` **(Enum)**: FSM state indices (`S0` through `S7`, total 8 states).
  - `StateStepper_t` **(Struct)**: FSM state representation structure.
    - `next[2]`: Stores next state indices based on rotation direction (`DIR_CW`, `DIR_CCW`).
    - `out[4]`: Stores digital output signals (0 or 1) for the 4 motor coils (A, B, A', B').
- **Function Prototypes**
  - `stepper_init()`: Initializes GPIO pins and driving mode.
  - `stepper_speed()`: Converts motor speed from RPM to step delay in milliseconds.
  - `stepper_step()`: Drives the motor for a specified number of steps and direction.
  - `stepper_out()`: Outputs digital signals to GPIO pins according to the current FSM state.
  - `stepper_stop()`: Cuts off power to all GPIO outputs to stop the motor.

```
/*----------------------------------------------------------------\
@ Embedded Controller by Young-Keun Kim - Handong Global University
Author           : Juyoung Lee
Created          : 05-03-2021
Modified         : 10-09-2026
Language/ver     : C++ in VS Code

Description      : Stepper Motor Library.
/----------------------------------------------------------------*/

#include "ecSTM32F4v2.h"
```

```
#ifndef __EC_STEPPER2_H
#define __EC_STEPPER2_H

#ifdef __cplusplus
 extern "C" {
#endif /* __cplusplus */

// Stepper Mode
#define HALF 0
#define FULL 1
#define DIR_CW 0
#define DIR_CCW 1

// Stepper Motor Spec
// 28BYJ-48 : 64 steps/rev x 1/32 gear = 2048
#define STEP_PER_REV  (64*32)

//State number
typedef enum {
    S0, S1, S2, S3, S4, S5, S6, S7
} StateNum;

// FULL stepping sequence  - FSM
typedef struct {
  uint32_t next[2];
  uint8_t out[4];
} StateStepper_t;

// Initialization of Stepper Motor
void stepper_init(uint32_t mode, PinName_t *pinStepper);

// Converts Motor [rpm] to step delay in [msec]
void stepper_speed (uint32_t speedRPM);

// Run Stepper Motor for given steps and direction
void stepper_step(uint32_t steps, uint32_t direction);

// Stepper Motor Output for given state
void stepper_out(uint32_t state);

// Stop Stepper Motor
void stepper_stop(void);


#ifdef __cplusplus
}
#endif /* __cplusplus */

#endif // __EC_STEPPER2_H
```

Preparation: ecStepper2.c

The `ecStepper2.c` file defines the FSM state tables and implements core control logic for driving the stepper motor.

- **Static & Volatile Variables**
  - `stepperPins[4]`: Array holding GPIO pin assignments for motor drive lines.
  - `_steps`: Total step count to be executed.
  - `_mode`: Active driving mode (`FULL` or `HALF`).
  - `_step_delay`: Inter-step delay time in milliseconds.
  - `*fsm`: Pointer to the selected FSM state lookup table (`fsm_Full` or `fsm_Half`).
  - `state`: Current FSM state index.
- **FSM State Tables**
  - `fsm_Full[4]`: 4-State Moore FSM table for Full-stepping (2-phase excitation).
  - `fsm_Half[8]`: 8-State Moore FSM table for Half-stepping (1-2 phase excitation for higher resolution).
- **Functions Description**
  - `stepper_init(uint32_t mode, PinName_t *pinStepper)`
    - Assigns the `fsm` pointer to `fsm_Full` or `fsm_Half` based on the input `mode`.
    - Configures the 4 designated GPIO pins as `OUTPUT`, `PUSH_PULL`, `NO_PUPD`, and `SPEED_FAST`.
  - `stepper_speed(uint32_t speedRPM)`
    - Calculates step delay (`_step_delay`, in ms) from target RPM:

      $$_step_delay = \frac{60 \times 1000}{\text{STEP_PER_REV} \times \text{speedRPM}}$$
    - In half-stepping mode, the step delay is halved.
  - `stepper_step(uint32_t steps, uint32_t direction)`
    - Calculates total step count (`_steps`) according to active mode (`steps` for `FULL`, `2 * steps` for `HALF`).
    - Iterates through step count, transitioning to the next state based on `direction`, executing `stepper_out()`, and applying `delay_ms(_step_delay)`.
  - `stepper_out(uint32_t state)`
    - Reads values from `out[0..3]` of the current FSM state and writes digital signals to the 4 GPIO pins (A, B, A', B') via `GPIO_write()`.
  - `stepper_stop(void)`
    - Clears remaining steps (`_steps = 0`) and sets all 4 GPIO pins to `0` (LOW) to cut coil current and halt motor rotation.

```
/*----------------------------------------------------------------\
```

```
@ Embedded Controller by Young-Keun Kim - Handong Global University
Author           : Juyoung Lee
Created          : 05-03-2021
Modified         : 10-09-2026
Language/ver     : C++ in VS Code

Description      : Stepper Motor Library.
/----------------------------------------------------------------*/

#include "ecStepper2.h"


// Stepper Motor variable
static PinName_t stepperPins[4];
static volatile uint32_t _steps;
static volatile uint32_t _mode;
static volatile uint32_t _step_delay = 100;
static const StateStepper_t *fsm;               // points at fsm_full or fsm_half (chosen in
stepper_init)
static volatile uint32_t state = 0;             // current FSM state



// Output: A B A' B'
static const StateStepper_t fsm_Full[4] = {
    {{S3,S1},{1,1,0,0}},        // A B A'B'
    {{S0,S2},{0,1,1,0}},        // A B A'B'
    {{S1,S3},{0,0,1,1}},        // A B A'B'
    {{S2,S0},{1,0,0,1}},        // A B A'B'
};

static const StateStepper_t fsm_Half[8] = {
    {{S7,S1},{1,0,0,0}},
    {{S0,S2},{1,1,0,0}},
    {{S1,S3},{0,1,0,0}},
    {{S2,S4},{0,1,1,0}},
    {{S3,S5},{0,0,1,0}},
    {{S4,S6},{0,0,1,1}},
    {{S5,S7},{0,0,0,1}},
    {{S6,S0},{1,0,0,1}},
};


// Initialization of Stepper Motor
void stepper_init(uint32_t mode, PinName_t *pinStepper){
    if (mode == HALF)
        fsm = fsm_Half;
    else
        fsm = fsm_Full;

    _mode = mode;

    //  GPIO Digital Out Initiation
```

```
    // For A, B, AN, BN
    for (int i=0; i<4; i++){
        stepperPins[i] = pinStepper[i];
        GPIO_init(stepperPins[i],OUTPUT);
        GPIO_pupd(stepperPins[i],NO_PUPD);
        GPIO_otype(stepperPins[i],PUSH_PULL);
        GPIO_ospeed(stepperPins[i],SPEED_FAST);
    }

}

// Converts Motor [rpm] to step delay in [msec]
void stepper_speed (uint32_t speedRPM){      // rpm
    if(_mode == FULL) _step_delay = 60*1000/(STEP_PER_REV*speedRPM);        //in [msec]
    else _step_delay =  60*1000/(STEP_PER_REV*speedRPM) / 2;                // delay of half
    // [usec] can be used for a higher accuracy
    // But needs delay_us()
}


// Run Stepper Motor for given steps and direction
void stepper_step(uint32_t steps, uint32_t direction){
    // Exit if stepper_init() not called

    if (fsm == 0) return;

    // Change step size
    if(_mode == FULL) _steps = steps;
    else _steps = 2 * steps;

    // run for step size:
    for (uint32_t i = 0; i < _steps; i++){

        // Next State
        state = fsm[state].next[direction];

        // Output of Present State
        stepper_out(state);

        // Delay for steppermotor speed
        delay_ms(_step_delay);
    }
}

// Stepper Motor Output for given state
void stepper_out(uint32_t state){

    // stepperPins[0]=A, stepperPins[1]=B, ...
    // out[0]=A, out[1]=B, ...

    GPIO_write(stepperPins[0], fsm[state].out[0]);
    GPIO_write(stepperPins[1], fsm[state].out[1]);
    GPIO_write(stepperPins[2], fsm[state].out[2]);
```

```
    GPIO_write(stepperPins[3], fsm[state].out[3]);
}


// Stop Stepper Motor
void stepper_stop(void) {
    _steps = 0;

    // All pins(A,AN,B,BN) set as DigitalOut '0'
    GPIO_write(stepperPins[0], 0);
    GPIO_write(stepperPins[1], 0);
    GPIO_write(stepperPins[2], 0);
    GPIO_write(stepperPins[3], 0);
}
```

Main Code

The main program initializes the system clock, external interrupts (EXTI), and stepper motor parameters, executing continuous bi-directional rotation within the infinite loop.

- **Global Variables**
  - `pinStepper[4] = {PB_10, PB_4, PB_5, PB_3}` : Defines STM32 GPIO pins connected to the motor driver inputs (IN1–IN4).
  - `motor_speed = 29` : Sets target motor rotation speed (29 RPM).
- **Functions Description**
  - `setup(void)`
    - `RCC_PLL_init()` : Configures main system clock to 84 MHz.
    - `SysTick_init()` : Initializes SysTick timer for 1 ms delay function (`delay_ms`).
    - `EXTI_init(SW1_PIN, FALL, 10)` : Configures user button (SW1) external interrupt on falling edge with priority level 10.
    - `stepper_init(HALF, pinStepper)` : Initializes stepper motor with Half-step mode and designated pins.
    - `stepper_speed(motor_speed)` : Applies initial motor speed (29 RPM).
  - `main(void)`
    - Calls `setup()` and enters the infinite loop (`while(1)`).
    - Calculates step count for 1 revolution (`steps = STEP_PER_REV * 1`).
    - Rotates counter-clockwise (`DIR_CCW`) for 1 revolution, delays 2 seconds (`delay_ms(2000)`), rotates clockwise (`DIR_CW`) for 1 revolution, and delays 2 seconds continuously.
  - `EXTI15_10_IRQHandler(void)` **(Interrupt Handler)**
    - Interrupt Service Routine triggered asynchronously when the user button (SW1) is pressed.
    - Checks for pending interrupt using `is_pending_EXTI(SW1_PIN)` and calls `stepper_stop()` to immediately stop motor operation.
    - Clears pending interrupt flag via `clear_pending_EXTI(SW1_PIN)` to prevent infinite ISR re-entry.

```
/*----------------------------------------------------------------\
@ Embedded Controller by Young-Keun Kim - Handong Global University
Author           : Juyoung Lee
Created          : 05-03-2021
Modified         : 10-01-2026
Language/ver     : C++ in VS Code

Description      : Stepper motor P1.Rotate the stepper motor 5 revolutions with 1 rpm.
/----------------------------------------------------------------*/

#include "ecSTM32F4v2.h"
```

```
// ( A, B,  AN,  BN)
PinName_t pinStepper[4]    = {PB_10,PB_4,PB_5,PB_3};
uint32_t motor_speed = 29;

// Initialiization
void setup(void){
    RCC_PLL_init();                                 // System Clock = 84MHz
    SysTick_init();                                 // Systick init

    EXTI_init(SW1_PIN, FALL, 10);

    stepper_init(HALF, pinStepper);                 // Stepper GPIO pin initialization
    stepper_speed(motor_speed);                                 //  set stepper motor speed
}


int main(void) {
    // Initialiization --------------------------------------------------------
    setup();
    uint32_t rev=1;
    uint32_t steps=STEP_PER_REV*rev;

    // Inifinite Loop ---------------------------------------------------------
    while(1){

        stepper_step(steps, DIR_CCW);   // (Step : 2048, Direction : 0 or 1, Mode : FULL or
HALF)
        delay_ms(2000);
        stepper_step(steps, DIR_CW);  // (Step : 2048, Direction : 0 or 1, Mode : FULL or
HALF)
        delay_ms(2000);

    }

}



void EXTI15_10_IRQHandler(void) {
    if (is_pending_EXTI(SW1_PIN)) {
        stepper_stop();

        clear_pending_EXTI(SW1_PIN); // cleared by writing '1'
    }
}
```

Results

1. Step Resolution Calculation

The total number of steps required for one full revolution (360°) of the 28BYJ-48 stepper motor in **Full-stepping mode** is calculated as follows:

$$\text{Steps}_{\text{Full}} = N_{\text{internal}} \times \text{Gear Ratio} = 64 \times 32 = 2048 \text{ steps/rev}$$

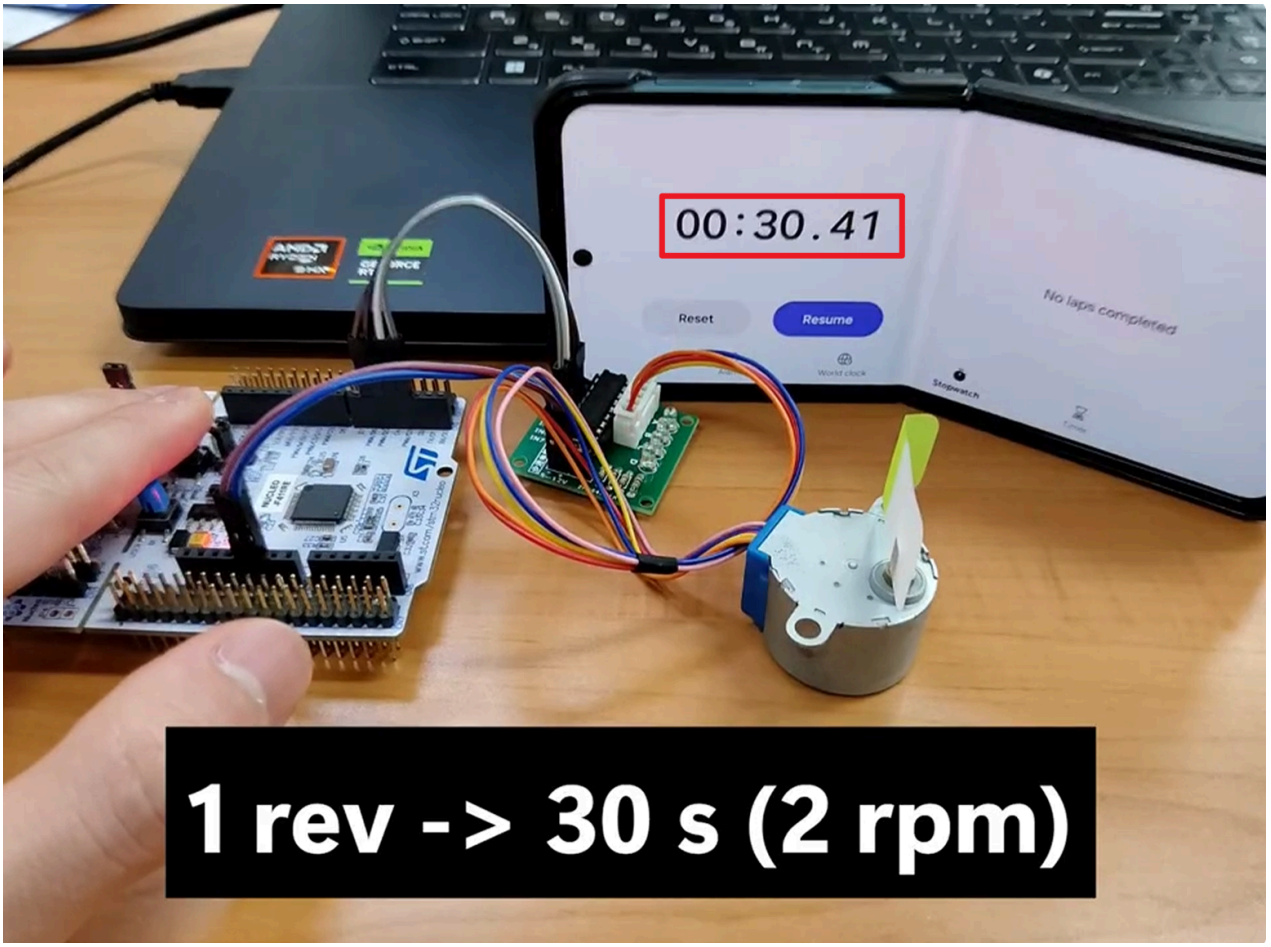
In **Half-stepping mode**, the angular resolution doubles, requiring twice the number of steps per revolution:

$$\text{Steps}_{\text{Half}} = \text{Steps}_{\text{Full}} \times 2 = 2048 \times 2 = 4096 \text{ steps/rev}$$

2. Then, rotate the stepper motor 1 revolutions with 2 rpm.

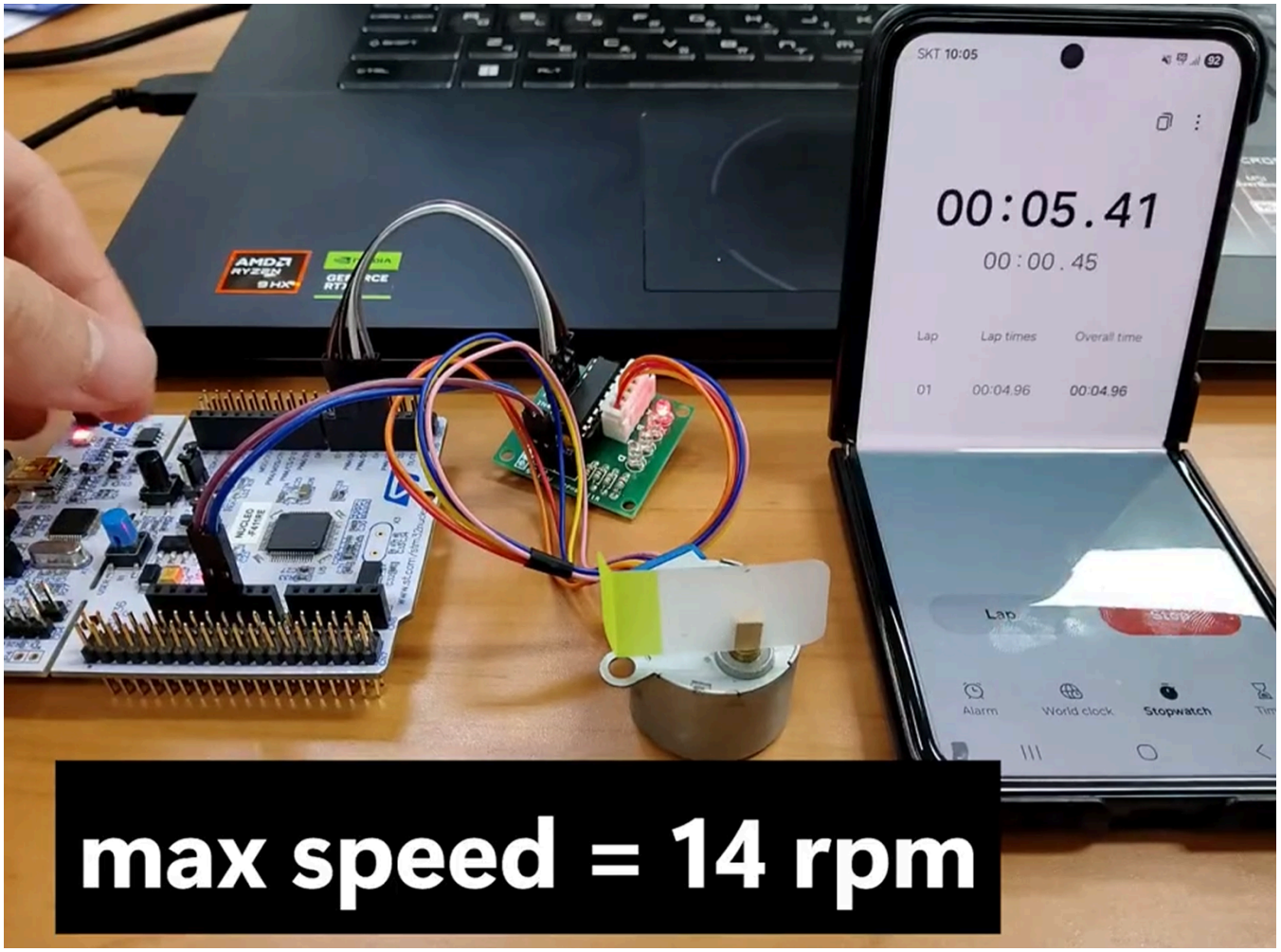
- **Test Condition**: Rotating the motor 1 revolution at a target speed of 2 RPM in full-stepping mode.
- **Theoretical Time**:

  $$\text{Time}_{\text{theoretical}} = \frac{60 \text{ s}}{2 \text{ RPM}} = 30.00 \text{ s}$$
- **Measured Time**: 30.41 s
- **Result Analysis**: The measured duration of 30.41 s matches the theoretical expected time (30.00 s) with minimal error, verifying that the step delay calculation and SysTick timing operating logic functioned correctly.



3. Increase and decrease the speed of the motor as fast as it can rotate to find the maximum and minimum speed of the motor.

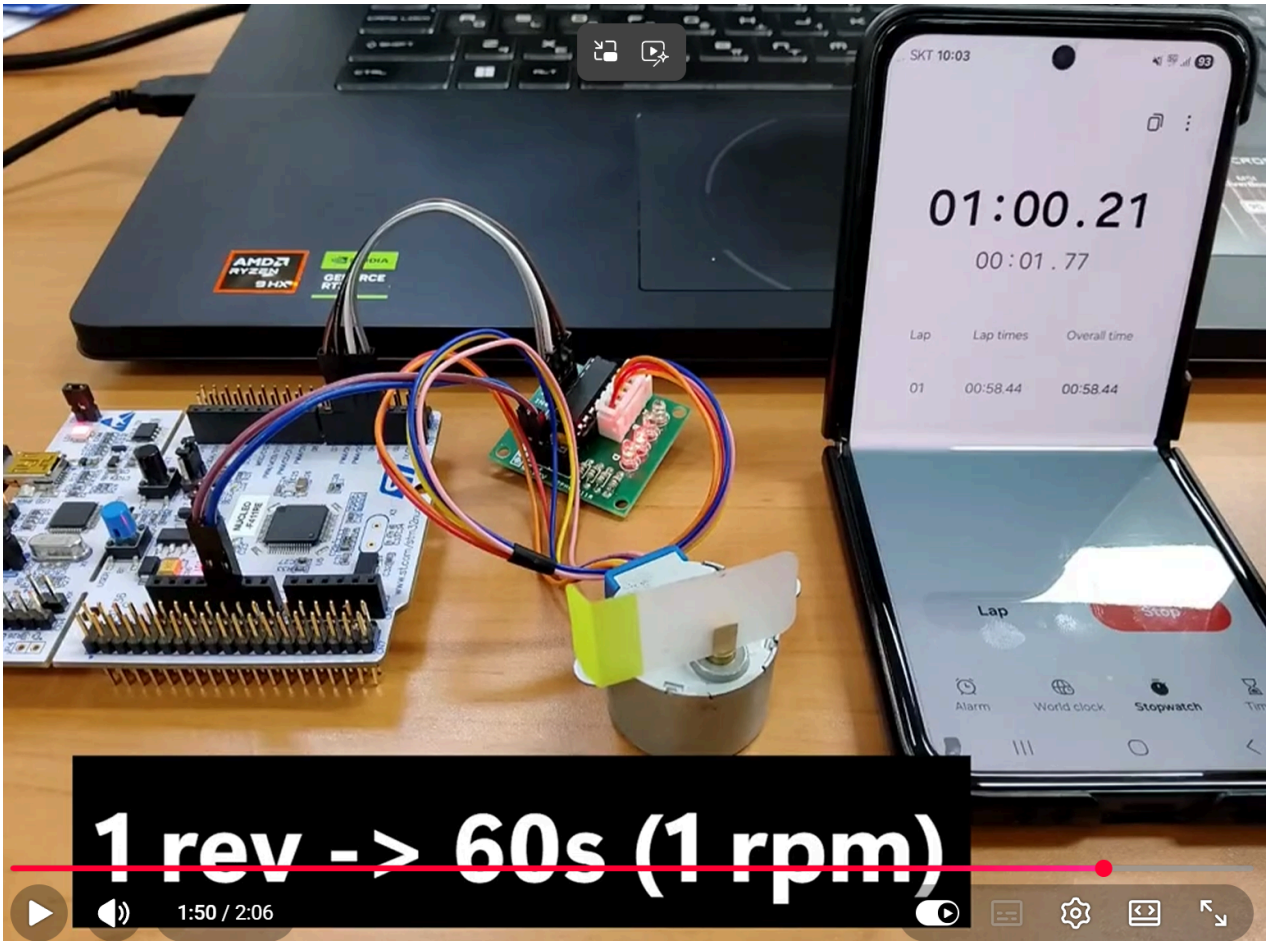
- **Maximum Achievable Speed**: **14 RPM**
- **Result Analysis**: When increasing the motor speed incrementally, the maximum stable operational speed without step loss was found to be 14 RPM. Attempting to drive the motor above 14 RPM caused the motor to stall (vibrate without turning) due to exceeding the pull-in/pull-out torque limit of the 28BYJ-48 motor under direct step excitation without a trapezoidal velocity ramp.



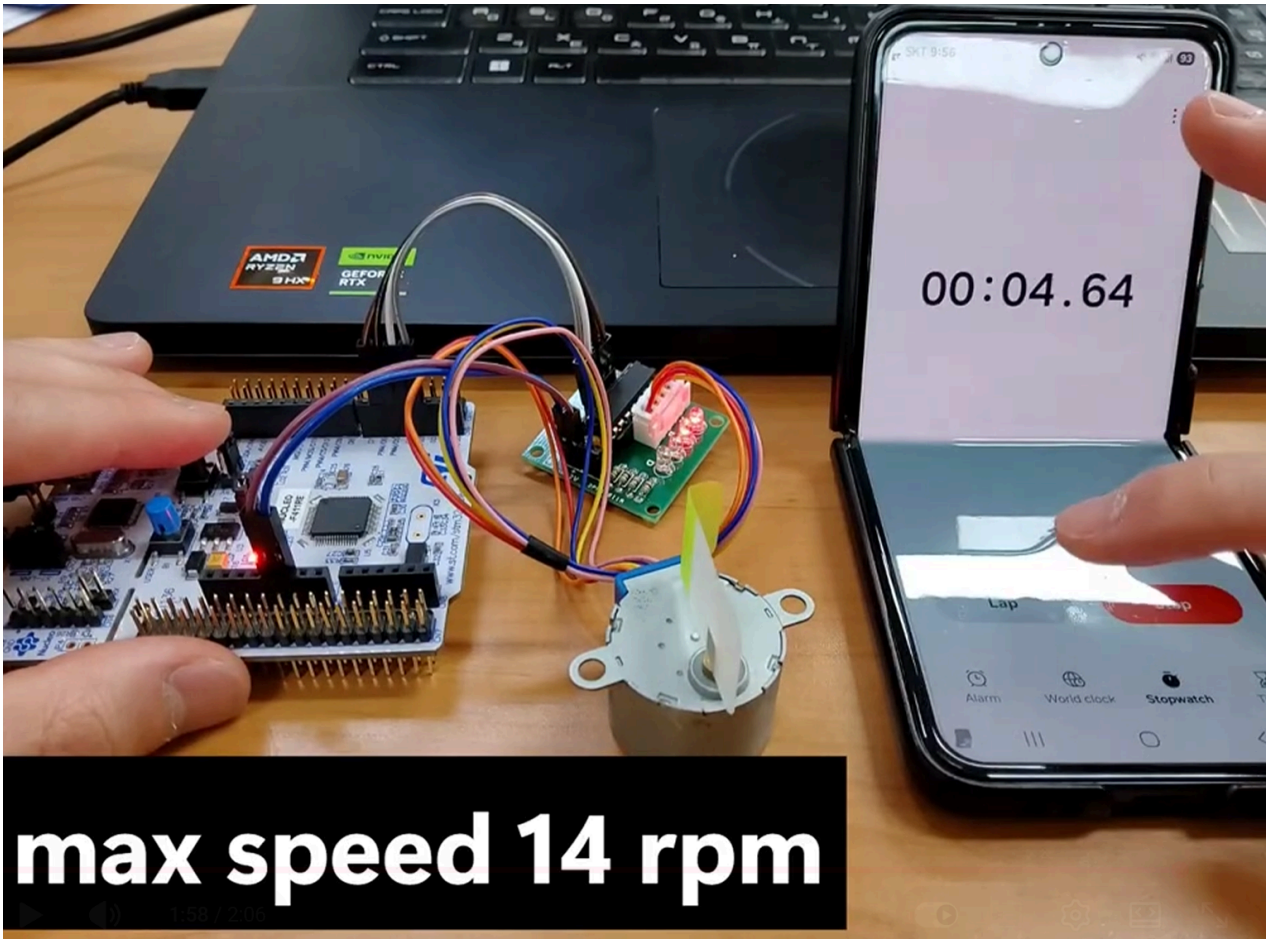
4. Apply the half-stepping and repeat the above.

Half-stepping doubled the step resolution and significantly reduced mechanical vibration during rotation. The maximum operation speed in half-stepping mode was also verified to be 14 RPM, as high pulse frequencies at higher speeds lead to torque degradation on the ULN2003 driver.

- **1 Revolution Test (at 1 RPM)**: The motor completed 1 full revolution ($4096\ \text{steps}$) smoothly at 1 RPM.



- **Maximum Achievable Speed**: **14 RPM**



Demo Video: [EC 26-2] LAB(Stepper Motor)

Discussion

1. Find out the trapezoid-shape velocity profile for a stepper motor. When is this profile necessary?

> The trapezoidal velocity profile is a motion control technique that manages motor speed over time in three distinct phases—acceleration, constant speed, and deceleration—which is essential for mitigating abrupt torque variations caused by load inertia (J). Instantly applying high-frequency step pulses from a standstill causes the required inertial torque ($T = J \cdot \alpha$) to exceed the motor's pull-in torque, resulting in lost steps (stall) and severe resonance noise; thus, an acceleration phase is necessary to reduce this initial inertial load. By incorporating these acceleration and deceleration ramps, the motor can safely bypass its initial self-start torque limit and reliably reach higher operating speeds within the pull-out torque region without stalling.

2. How would you change the code more efficiently for micro-stepping control? You don't have to code this but need to explain your strategy.

> To efficiently implement micro-stepping control, a hardware-centered architecture is required to minimize CPU processing overhead and system latency. Instead of relying on blocking software delays like `delay_ms()`, pairing STM32 hardware timer PWM with Direct Memory Access (DMA) enables autonomous waveform generation without CPU intervention, while referencing a pre-computed Look-Up Table (LUT) for PWM duty cycles at each micro-step position eliminates real-time trigonometric calculation overhead. Furthermore, offloading the control logic to a dedicated hardware indexer driver that requires only two control lines (STEP and DIR)—rather than having the MCU generate four-phase PWM signals directly—is the most effective strategy to conserve MCU computational and I/O resources.

3. There are other types of Stepper Motor Drivers that are simple to use, such as you only give one pulse signal and direction, instead of giving 4 pulse signals. Such examples are A4988, DRV 8834, and TB6600 drivers. Compare these motor drivers with ULN2003 in terms of operating method.

> The ULN2003 is a voltage-controlled switching device based on a simple Darlington transistor array, requiring the MCU to directly toggle four GPIO pins (IN1–IN4) according to a software FSM state table, which results in high heat generation, low power efficiency, and limited micro-stepping capability. In contrast, dedicated drivers such as the A4988, DRV8834, and TB6600 feature an internal hardware translator and H-Bridge MOSFET constant-current chopping circuit, allowing them to receive only two control inputs (STEP pulse and DIR direction) from the MCU to handle micro-stepping division and coil current regulation with high torque output and superior energy efficiency.

Feature	ULN2003	Dedicated Drivers (A4988, etc.)
Control Method	**4-Wire Phase Control** (IN1–IN4)	**2-Wire STEP / DIR Control**

Feature	ULN2003	Dedicated Drivers (A4988, etc.)
FSM Execution	**MCU (Software)** Requires manual sequence step toggling	**Internal Translator (Hardware)** Translates STEP pulse and DIR signals directly
Driving Method	Darlington Transistor Array (Voltage Control)	H-Bridge MOSFET + Constant-Current Chopping
Micro-stepping	Unsupported (Limited to Full/Half stepping)	Supports hardware micro-stepping up to 1/16 or 1/32
Power & Efficiency	High voltage drop, high heat generation, low efficiency	Supports high power supply (12–24V), high torque, low heat

Reference

[1] STMicroelectronics, *STM32F411xC/E Reference Manual (RM0383)*, Rev 4, 2025. [Online]. Available: STMicroelectronics RM0383

[2] STMicroelectronics, "ULN2003: Seven Darlington Arrays Datasheet," [Online]. Available: https://www.alldatasheet.co.kr/datasheet-pdf/pdf/25575/STMICROELECTRONICS/ULN2003.html. [Accessed: Oct. 12, 2026].